

```
-- =====  
  
-- SWIGGY FOOD ORDERS - END TO END SQL ANALYSIS  
  
-- Database: swiggy_db | Table: swiggy_orders  
  
-- 197,430 records | Jan 2025 - Aug 2025  
  
-- =====
```

```
-- _____  
  
-- TABLE CREATION & SETUP  
  
-- _____
```

```
CREATE DATABASE swiggy_db;
```

```
USE swiggy_db;
```

```
DROP TABLE IF EXISTS swiggy_orders;
```

```
CREATE TABLE swiggy_orders (  
    order_id      INT AUTO_INCREMENT PRIMARY KEY,  
    state         VARCHAR(100),  
    city          VARCHAR(100),  
    order_date    DATE,  
    restaurant_name VARCHAR(200),  
    location      VARCHAR(200),  
    category      VARCHAR(200),  
    dish_name     VARCHAR(300),  
    price_inr     DECIMAL(10, 2),
```

```

rating            DECIMAL(3, 1),
rating_count      INT,
food_type         VARCHAR(10) GENERATED ALWAYS AS (
                    CASE
                        WHEN LOWER(dish_name) REGEXP
'chicken|mutton|fish|egg|prawn|lamb|beef|pork|seafood|biryani|kebab'
                        THEN 'Non-Veg' ELSE 'Veg'
                    END
                ) STORED,
order_month       INT GENERATED ALWAYS AS (MONTH(order_date)) STORED,
order_quarter     INT GENERATED ALWAYS AS (QUARTER(order_date)) STORED,
day_of_week       VARCHAR(10) GENERATED ALWAYS AS (DAYNAME(order_date)) STORED
);

-- LOAD DATA (adjust path as needed)

-- LOAD DATA INFILE '/path/to/swiggy_data.csv'

-- INTO TABLE swiggy_orders FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\n'

-- IGNORE 1 ROWS (state, city, order_date, restaurant_name, location, category, dish_name, price_inr,
rating, rating_count);

-- =====

-- QUERY 1: OVERALL KPI SUMMARY DASHBOARD

-- Top-level business metrics for the executive summary card

-- =====

```

```

SELECT
    CONCAT('₹', FORMAT(SUM(price_inr), 0))          AS total_sales,
    FORMAT(COUNT(*), 0)                             AS total_orders,
    CONCAT('₹', FORMAT(AVG(price_inr), 2))          AS avg_order_value,
    FORMAT(AVG(rating), 2)                          AS avg_rating,
    FORMAT(SUM(rating_count), 0)                    AS total_ratings_count,
    COUNT(DISTINCT restaurant_name)                AS unique_restaurants,
    COUNT(DISTINCT city)                           AS cities_covered,
    COUNT(DISTINCT state)                          AS states_covered
FROM swiggy_orders;

```

/*

Expected Output:

total_sales	total_orders	avg_order_val	avg_rating	total_ratings
₹53,012,505.77	197,430	₹268.51	4.34	5,593,929

*/

-- =====

-- QUERY 2: MONTHLY SALES TREND

-- Tracks total revenue and order volume month over month

-- =====

SELECT

order_month,

MONTHNAME(STR_TO_DATE(order_month, '%m')) AS month_name,

COUNT(*) AS total_orders,

ROUND(SUM(price_inr), 2) AS total_sales,

ROUND(AVG(price_inr), 2) AS avg_order_value,

ROUND(AVG(rating), 2) AS avg_rating,

ROUND(

(SUM(price_inr) - LAG(SUM(price_inr)) OVER (ORDER BY order_month))

/ LAG(SUM(price_inr)) OVER (ORDER BY order_month) * 100, 2

) AS mom_growth_pct

FROM swiggy_orders

GROUP BY order_month

ORDER BY order_month;

/*

MoM Growth shows sales peaked in Jan (₹68.25L) with minor dip in Feb,

recovering steadily through Q2 and Q3.

*/

-- =====

-- QUERY 3: QUARTERLY PERFORMANCE SUMMARY

-- Sales, Ratings, and Orders grouped by business quarter

-- =====

SELECT

CONCAT('Q', order_quarter)	AS quarter,
COUNT(*)	AS total_orders,
ROUND(SUM(price_inr) / 100000, 2)	AS total_sales_lakhs,
ROUND(AVG(price_inr), 2)	AS avg_order_value,
ROUND(AVG(rating), 2)	AS avg_rating,
SUM(rating_count)	AS total_ratings,
ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER(), 2)	AS order_share_pct

FROM swiggy_orders

GROUP BY order_quarter

ORDER BY order_quarter;

/*

Q1: ₹196.68L | 73.1K orders | 4.3 avg rating

Q2: ₹198.02L | 74.2K orders | 4.3 avg rating

Q3 (Jan–Aug): ₹134.4L | 50.2K orders (partial quarter)

*/

-- =====

-- QUERY 4: DAILY SALES TREND (DAY OF WEEK ANALYSIS)

-- Identifies peak and off-peak ordering days

-- =====

SELECT

day_of_week,

FIELD(day_of_week,

'Sunday','Monday','Tuesday','Wednesday','Thursday','Friday','Saturday'

) AS day_order,

COUNT(*) AS total_orders,

ROUND(SUM(price_inr), 2) AS total_sales,

ROUND(AVG(price_inr), 2) AS avg_order_value,

ROUND(AVG(rating), 2) AS avg_rating,

ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER(), 2) AS orders_share_pct

FROM swiggy_orders

GROUP BY day_of_week

ORDER BY day_order;

/*

Saturday and Sunday see highest order volumes (+12% vs weekday avg),

while Tuesday is the lowest performing day.

*/

-- =====

-- QUERY 5: TOP 10 CITIES BY TOTAL SALES

-- Ranks cities by revenue contribution with running total

-- =====

WITH city_sales AS (

SELECT

city,

state,

COUNT(*) AS total_orders,

ROUND(SUM(price_inr), 2) AS total_sales,

ROUND(AVG(price_inr), 2) AS avg_order_value,

ROUND(AVG(rating), 2) AS avg_rating

FROM swiggy_orders

GROUP BY city, state

),

ranked AS (

SELECT *,

RANK() OVER (ORDER BY total_sales DESC) AS sales_rank,

ROUND(total_sales * 100.0 / SUM(total_sales) OVER(), 2) AS revenue_share_pct,

ROUND(SUM(total_sales) OVER (ORDER BY total_sales DESC

ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)

* 100.0 / SUM(total_sales) OVER(), 2) AS cumulative_pct

FROM city_sales

)

SELECT * FROM ranked

ORDER BY sales_rank

LIMIT 10;

/*

Top 5: Bengaluru (₹54.6L) > Mumbai > Hyderabad > Jaipur > Lucknow

Top 5 cities contribute ~54% of total platform revenue.

*/

-- =====

-- QUERY 6: STATE-WISE SALES DISTRIBUTION

-- Geographic revenue breakdown across all 28 states

-- =====

SELECT

state,

COUNT(DISTINCT city)

AS cities,

COUNT(*)

AS total_orders,

ROUND(SUM(price_inr) / 100000, 2)

AS total_sales_lakhs,

ROUND(AVG(price_inr), 2)

AS avg_order_value,

ROUND(AVG(rating), 2)

AS avg_rating,

ROUND(SUM(price_inr) * 100.0 / SUM(SUM(price_inr)) OVER(), 2) AS revenue_share_pct

FROM swiggy_orders

GROUP BY state

ORDER BY total_sales_lakhs DESC;

/*

Karnataka leads with ₹54.6L (10.3% share), followed by

Uttar Pradesh ₹31.2L and Telangana ₹30.2L.

*/

-- =====

-- QUERY 7: VEG vs NON-VEG SALES COMPARISON

-- Revenue split by food type with per-city breakdown

-- =====

SELECT

food_type,

COUNT(*) AS total_orders,

ROUND(SUM(price_inr), 2) AS total_sales,

ROUND(AVG(price_inr), 2) AS avg_price,

ROUND(AVG(rating), 2) AS avg_rating,

SUM(rating_count) AS total_reviews,

ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER(), 2) AS order_share_pct,

ROUND(SUM(price_inr) * 100.0 / SUM(SUM(price_inr)) OVER(), 2) AS revenue_share_pct

FROM swiggy_orders

GROUP BY food_type;

/*

Veg: 139,819 orders | ₹337.8L (63.7% revenue) | Avg ₹241.6

Non-Veg: 57,611 orders | ₹192.3L (36.3% revenue) | Avg ₹333.9

(Non-Veg has higher AOV despite fewer orders)

*/

-- =====

-- QUERY 8: TOP 5 RESTAURANTS BY REVENUE

-- Best performing restaurants with performance metrics

-- =====

WITH restaurant_stats AS (

SELECT

restaurant_name,

city,

state,

COUNT(*) AS total_orders,

ROUND(SUM(price_inr), 2) AS total_revenue,

ROUND(AVG(price_inr), 2) AS avg_order_value,

ROUND(AVG(rating), 2) AS avg_rating,

SUM(rating_count) AS total_reviews,

COUNT(DISTINCT dish_name) AS menu_items

FROM swiggy_orders

GROUP BY restaurant_name, city, state

),

ranked_restaurants AS (

```

SELECT *,
        DENSE_RANK() OVER (ORDER BY total_revenue DESC) AS revenue_rank
FROM restaurant_stats
)
SELECT * FROM ranked_restaurants
WHERE revenue_rank <= 5
ORDER BY revenue_rank;

```

```

/*
#1 KFC          ₹42.5L | 15,830 orders
#2 McDonald's  ₹33.4L | 12,890 orders
#3 Pizza Hut    ₹21.3L | 8,140 orders
#4 Burger King ₹19.0L | 7,620 orders
#5 Domino's     ₹18.3L | 7,100 orders
*/

```

```

-- =====
-- QUERY 9: WEEKLY SALES TREND ANALYSIS
-- Week-by-week revenue to identify peak and slow periods
-- =====

```

```

SELECT
    YEAR(order_date)                AS year,
    WEEK(order_date, 1)             AS week_number,

```

```

MIN(order_date)                AS week_start,
MAX(order_date)                AS week_end,
COUNT(*)                     AS total_orders,
ROUND(SUM(price_inr), 2)       AS weekly_sales,
ROUND(AVG(price_inr), 2)       AS avg_order_value,
ROUND(
    (SUM(price_inr) - LAG(SUM(price_inr)) OVER (ORDER BY WEEK(order_date, 1)))
    / LAG(SUM(price_inr)) OVER (ORDER BY WEEK(order_date, 1)) * 100, 2
)                               AS wow_growth_pct
FROM swiggy_orders
GROUP BY YEAR(order_date), WEEK(order_date, 1)
ORDER BY week_number;

```

```
-- =====
```

```
-- QUERY 10: CUSTOMER SATISFACTION ANALYSIS
```

```
-- Rating distribution and quality segmentation
```

```
-- =====
```

```
SELECT
```

```
    CASE
```

```
        WHEN rating >= 4.5 THEN '★ Excellent (4.5–5.0)'
```

```
        WHEN rating >= 4.0 THEN '✔ Good (4.0–4.4)'
```

```
        WHEN rating >= 3.5 THEN '⚠ Average (3.5–3.9)'
```

```
        ELSE '✖ Poor (Below 3.5)'
```

```

        END                                AS rating_segment,
        COUNT(*)                          AS order_count,
        ROUND(SUM(price_inr), 2)           AS total_revenue,
        ROUND(AVG(price_inr), 2)           AS avg_order_value,
        ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER(), 2) AS pct_of_orders,
        ROUND(SUM(price_inr) * 100.0 / SUM(SUM(price_inr)) OVER(), 2) AS pct_of_revenue
FROM swiggy_orders
GROUP BY rating_segment
ORDER BY MIN(rating) DESC;

```

/*

Excellent: 81.4% of orders → Premium satisfaction drives bulk revenue

Good: 14.7% of orders

Average: 3.1% of orders

Poor: 0.8% of orders → Focus area for restaurant quality improvement

*/

-- =====

-- QUERY 11: COHORT ANALYSIS - MONTHLY REVENUE PER CITY

-- Pivot-style breakdown of city performance month by month

-- =====

SELECT

city,

```

ROUND(SUM(CASE WHEN order_month = 1 THEN price_inr ELSE 0 END), 0) AS jan_sales,
ROUND(SUM(CASE WHEN order_month = 2 THEN price_inr ELSE 0 END), 0) AS feb_sales,
ROUND(SUM(CASE WHEN order_month = 3 THEN price_inr ELSE 0 END), 0) AS mar_sales,
ROUND(SUM(CASE WHEN order_month = 4 THEN price_inr ELSE 0 END), 0) AS apr_sales,
ROUND(SUM(CASE WHEN order_month = 5 THEN price_inr ELSE 0 END), 0) AS may_sales,
ROUND(SUM(CASE WHEN order_month = 6 THEN price_inr ELSE 0 END), 0) AS jun_sales,
ROUND(SUM(CASE WHEN order_month = 7 THEN price_inr ELSE 0 END), 0) AS jul_sales,
ROUND(SUM(CASE WHEN order_month = 8 THEN price_inr ELSE 0 END), 0) AS aug_sales,
ROUND(SUM(price_inr), 0) AS grand_total
FROM swiggy_orders
GROUP BY city
ORDER BY grand_total DESC;

```

```

-- =====
-- QUERY 12: PRICE BAND ANALYSIS
-- Segments orders into price buckets to identify demand curve
-- =====

```

```

SELECT
CASE
    WHEN price_inr < 100 THEN '🎯 Budget (< ₹100)'
    WHEN price_inr < 250 THEN '📦 Economy (₹100–249)'
    WHEN price_inr < 500 THEN '📦 Mid-range (₹250–499)'
    WHEN price_inr < 1000 THEN '🎯 Premium (₹500–999)'

```

```

        ELSE
            '💎 Luxury (₹1000+)'
    END
    AS price_band,
    COUNT(*)
    AS total_orders,
    ROUND(SUM(price_inr), 2)
    AS total_revenue,
    ROUND(AVG(rating), 2)
    AS avg_rating,
    ROUND(COUNT(*) * 100.0 / SUM(COUNT(*) OVER(), 2) AS order_share_pct
FROM swiggy_orders
GROUP BY price_band
ORDER BY MIN(price_inr);

```

/*

Economy (₹100–249) dominates with 42% of all orders

Budget (<₹100) surprisingly has higher avg rating (4.38) than Premium segment

*/

```
-- =====
```

```
-- BONUS: STORED PROCEDURE - CITY PERFORMANCE REPORT
```

```
-- Run: CALL GetCityReport('Bengaluru');
```

```
-- =====
```

DELIMITER

```
CREATE PROCEDURE GetCityReport(IN p_city VARCHAR(100))
```

```
BEGIN
```

```

SELECT
    restaurant_name,
    COUNT(*)                AS orders,
    ROUND(SUM(price_inr), 2) AS revenue,
    ROUND(AVG(rating), 2)   AS avg_rating,
    food_type
FROM swiggy_orders
WHERE city = p_city
GROUP BY restaurant_name, food_type
ORDER BY revenue DESC
LIMIT 20;
END$$

```

```

DELIMITER ;

```

```

-- =====

```

```

-- END OF SQL ANALYSIS

```

```

-- Total Queries: 12 + 1 Stored Procedure

```

```

-- Author: Data Analytics Project | Swiggy 2025

```

```

-- =====

```

```

(DAX(MONTH(DateTable[Date]))

```

```

Month Name = FORMAT(DateTable[Date], "MMM")

```

```

Quarter = "Q" & QUARTER(DateTable[Date])

```

```

Day Name = FORMAT(DateTable[Date], "dddd")

```

```

WeekNumber = WEEKNUM(DateTable[Date])).

```